

```

1  (*
2
3      CS51 Lab 8
4      Functors
5      Part 2
6
7  *)
8
9      SOLUTION
10
11 (* Objective:
12
13 This lab provides practice in the use of functors.
14 *)
15
16 (*=====
17 Functors - Part 2.
18
19 In this second part of the lab, you'll rewrite the `Stack` module from
20 the previous lab so that its element type is polymorphic, while adding
21 some functionality at the same time.
22
23 Occasionally, it's useful to be able to take a data structure and
24 "serialize" it (also called "marshalling"), that is, to transform its
25 internal computational representation into a format, like a string,
26 that can be saved in a file or transmitted digitally. It can later be
27 "deserialized" to recreate the internal representation. We'll add to
28 the stack module extremely basic support for serialization.
29
30 (You may notice that this provides an opportunity to break an
31 abstraction barrier: serializing an object, manipulating its string
32 representation, and then deserializing back to an object could
33 allow for some invariant to be violated. Even so, serialization has
34 its uses when the circumstances warrant.)
35
36 In order to serialize stacks from an abstract stack data type, we will
37 need to have access to a function that can serialize the *elements* of
38 the stack.
39
40 We can use a functor that takes as argument a module for the stack
41 elements (including element serialization functionality) and generates
42 a stack module that bundles together all of the stack functionality,
43 including serialization.
44
45 In order to do this, we'll first define a module interface, called
46 SERIALIZE, that captures the requirements for the elements of a
47 stack. It requires that the module expose both a data type and a
48 function (`serialize`) that converts values of that type into a string
49 representation. The SERIALIZE module type is an appropriate signature
50 for the argument of a functor that generates serializable stack
51 modules. *)
52

```

```

53 module type SERIALIZE =
54   sig
55     type t
56     val serialize : t -> string
57   end ;;
58
59 (* Now we'll define a STACK interface. Notice that unlike the
60 `INT_STACK` interface from the previous lab, we'll specify the
61 `element` type to be an abstract type as part of the signature, and
62 add additional functions for serialization, as well as a couple of
63 higher-order functions over stacks. *)
64
65 module type STACK =
66   sig
67     exception Empty
68     type element
69     type stack
70     val empty : stack
71     val push : element -> stack -> stack
72     val top : stack -> element
73     val pop : stack -> stack
74     val serialize : stack -> string
75     val map : (element -> element) -> stack -> stack
76     val filter : (element -> bool) -> stack -> stack
77     val fold_left : ('a -> element -> 'a) -> 'a -> stack -> 'a
78   end ;;
79
80 (*.....
81 Exercise 1A: Rewrite your stack implementation from the last lab to
82 use the `element` abstract data type for elements, and also complete
83 the other functions that the signature requires. We've provided a good
84 start below.
85
86 The serialize function should construct a string in the following
87 form:
88
89     "N0:N1:N2:...:N"
90
91 where N0 was the *first* element pushed onto the stack, N1 was the
92 *second*, and so on, with N being the *most recent* element pushed to
93 the stack, that is, the one to be popped next. (This would make
94 deserialization easier, since elements could be pushed as soon as they
95 were read.)
96
97 (Don't forget that you can implement other functions that are not
98 specified by the signature if you would like. The module signature
99 will act as an abstraction barrier to prevent the extra functions from
100 leaking out of the module.)
101 .....*)
102
103 module MakeStack (Element: SERIALIZE)
104   : (STACK with type element = Element.t) =
105   struct
106     exception Empty

```

```

107
108     type element = Element.t
109     type stack = element list
110
111     let empty : stack = []
112
113     let push (el : element) (s : stack) : stack =
114         el :: s
115
116     let pop_helper (s : stack) : (element * stack) =
117         match s with
118         | [] -> raise Empty
119         | h :: t -> (h, t)
120
121     let top (s : stack) : element =
122         fst (pop_helper s)
123
124     let pop (s : stack) : stack =
125         snd (pop_helper s)
126
127     let map : (element -> element) -> stack -> stack =
128         List.map
129
130     let filter : (element -> bool) -> stack -> stack =
131         List.filter
132
133     let fold_left : ('a -> element -> 'a) -> 'a -> stack -> 'a =
134         List.fold_left
135
136     let serialize (s : stack) : string =
137         let string_join x y =
138             Element.serialize y
139             ^ (if x <> "" then ":" ^ x else "") in
140         fold_left string_join "" s
141     end ;;
142
143     (* .....
144     Exercise 1B: Now, make a module `IntStack` by applying the functor
145     that you just defined to an appropriate module for serializing integers.
146     .....*)
147
148     (* In the following solution, we are explicit about all module types,
149     while providing appropriate sharing constraints. *)
150
151     module IntSerialize : (SERIALIZE with type t = int) =
152     struct
153         type t = int
154         let serialize = string_of_int
155     end ;;
156
157     module IntStack : (STACK with type element = IntSerialize.t) =
158         MakeStack (IntSerialize) ;;
159
160     (* It might be a good exercise to drop one or another of the sharing

```

```

161     constraints, or of the module typings, and see what happens. *)
162
163 (*.....
164 Exercise 1C: Make a module `IntStringStack` that creates a stack whose
165 elements are `int * string` pairs. Its serialize function should output
166 elements as strings of the form:
167
168     "(N, 'S')"
```

where N is the int, and S is the string. For instance, a stack with two elements might be serialized as the string

```

172     "(1, 'pushed first'):(2, 'pushed second')"
```

For this oversimplified serialization function, you may assume that the string will be made up of alphanumeric characters only.

```

177 .....*)
178
179 (* This time, we left off the typings for the modules
180 `IntStringSerialize` and `IntStringStack`. Can you add them in such
181 a way that the modules are properly abstracted but can still be
182 used to construct and manipulate stacks? *)
183
184 module IntStringSerialize =
185   struct
186     type t = int * string
187     let serialize (n, s) =
188       "(" ^ string_of_int n ^ ", '" ^ s ^ "'"
189   end ;;
190
191 module IntStringStack =
192   MakeStack(IntStringSerialize) ;;

```